

Z LANGUAGE

Reference Manual — Documentazione della Sintassi

Tipi · Variabili · Array · Matrici · Operatori · Funzioni · I/O · Librerie

1 · Introduzione al Linguaggio Z

Z è un linguaggio interpretato con sintassi **token-based**. Ogni riferimento a un dato (variabile, array, numero, carattere) è racchiuso tra il simbolo delimitatore & nella forma generale:

```
&tipo;&nome;& oppure &tipo;[indice]&nome;& oppure &tipo;[r][c]&nome;&
```

Ogni istruzione termina con il carattere : (eccetto le aperture/chiusure di blocchi { }). Il punto di ingresso del programma è la funzione speciale `_start()`. I commenti si scrivono con `//`.

2 · Sistema dei Tipi

Il tipo di un dato è sempre indicato dal primo carattere all'interno dei delimitatori &. La tabella seguente riassume tutti i tipi disponibili.

Tipo	Lettera	Usabile per	Esempio token	Note
Intero	i	variabili, array, matrici, immediati	&i;&nome;&	Valore int C (32-bit)
Float	l	variabili, array, matrici	&l;&nome;&	Valore float C
Char singolo	c	variabili scalari	&c;&nome;&	Un solo carattere
Char immediato	k	sorgente in assegnazioni	&k;&X;&	Valore char letterale (non dichiarabile)
Numero immediato	n	sorgente in assegnazioni	&n;&42&	Numero intero letterale (non dichiarabile)
Array char/str	s	array, matrici di caratteri	&s;[5]&nome;&	Stringa o array di char
File	f	import di librerie	&f;&lib.Zlib;&	NON IMPLEMENTATO

3 · Sintassi degli Indici

Quando si accede a un array o a una matrice, l'indice può essere espresso in tre modi diversi:

Tipo indice	Sintassi	Esempio	Nota
Numero intero letterale	N	&i;[3]&arr;&	Accede all'elemento 3 di arr
Registro	rN	&i;[r2]&arr;&	Usa il valore del registro r2 come indice
Variabile intera	&i;&nome;_var&	&i;[&i;&idx;&]&arr;&	Usa il valore della variabile intera idx
Vuoto (stringa intera)	(vuoto)	&s;[]&str;&	Solo per char array: punta all'inizio

4 · Dichiarazioni di Variabili e Array

Tutte le dichiarazioni terminano con `;`. Le variabili vengono inizializzate a 0 (int/float) o a `'\0'` (char).

Costrutto	Sintassi	Esempio	Descrizione
Variabile int	<code>int_ &i;&nome;&;</code>	<code>int_ &i;&contatore;&;</code>	Dichiara variabile intera
Array int	<code>int_ &i;&[dim]&nome;&;</code>	<code>int_ &i;&[10]&numeri;&;</code>	Dichiara array di 10 interi
Variabile char	<code>char_ &c;&nome;&;</code>	<code>char_ &c;&lettera;&;</code>	Dichiara variabile char singola
Array char/str	<code>char_ &s;&[dim]&nome;&;</code>	<code>char_ &s;&[20]&parola;&;</code>	Dichiara array di 20 char (stringa)

■ La dimensione *dim* può essere un numero letterale, una variabile (`&i;&var;&`) o un registro (*rN*). Le matrici si dichiarano con due indici: `int_ &i;&[r][c]&nome;&;`

5 · Operatore di Assegnazione =

L'assegnazione copia il valore della **sorgente** nella **destinazione**. La destinazione non può essere un numero immediato (`&n;&`). I tipi devono essere compatibili (vedi Sezione 2).

Forma generale:

```
= &tipo;_dest&dest;& &tipo;_src&src;& ;
```

Caso	Sintassi completa	Descrizione
<code>N → var int</code>	<code>= &i;&var;& &n;&42& ;</code>	Assegna il numero 42 alla variabile var
<code>var → var</code>	<code>= &i;&v1;& &i;&v2;& ;</code>	Copia v2 in v1
<code>arr[idx] → var</code>	<code>= &i;&v;& &i;&[3]&arr;& ;</code>	Copia arr[3] in v
<code>var → arr[idx]</code>	<code>= &i;&[3]&arr;& &i;&v;& ;</code>	Copia v in arr[3]
<code>arr[a] → arr[b]</code>	<code>= &i;&[2]&a;& &i;&[0]&b;& ;</code>	Copia b[0] in a[2]
<code>N → arr[idx]</code>	<code>= &i;&[1]&arr;& &n;&7& ;</code>	Assegna 7 a arr[1]
<code>K → char var</code>	<code>= &c;&ch;& &k;&H;& ;</code>	Assegna il carattere 'H' a ch
<code>char var → char var</code>	<code>= &c;&ch1;& &c;&ch2;& ;</code>	Copia ch2 in ch1
<code>K → carr[idx]</code>	<code>= &s;&[0]&str;& &k;&Z;& ;</code>	Assegna 'Z' a str[0]
<code>carr[i] → carr[j]</code>	<code>= &s;&[2]&a;& &s;&[0]&b;& ;</code>	Copia b[0] in a[2]
<code>carr[i] → char var</code>	<code>= &c;&ch;& &s;&[1]&str;& ;</code>	Copia str[1] in ch
<code>func → var</code>	<code>= &i;&v;& &__fn(args) ;</code>	Assegna il valore di ritorno di fn a v

6 · Operatori Matematici

Le operazioni matematiche hanno il simbolo dell'operatore come **prefisso**. La forma con tre operandi scrive il risultato nella destinazione; la forma con due operandi modifica direttamente la destinazione.

Operatore	Forma 3 operandi (risultato → dest)	Forma 2 operandi (modifica dest)	Nota
+ somma	= + &i;&dest;& &i;&a;& &i;&b;& :	+ &i;&dest;& &i;&b;& :	dest = dest + b (forma 2)
- sottrazione	= - &i;&dest;& &i;&a;& &i;&b;& :	- &i;&dest;& &i;&b;& :	dest = dest - b
* moltiplicaz.	= * &i;&dest;& &i;&a;& &i;&b;& :	* &i;&dest;& &i;&b;& :	dest = dest * b
/ divisione	= / &i;&dest;& &i;&a;& &i;&b;& :	/ &i;&dest;& &i;&b;& :	dest = dest / b (intera)

■ Al posto di variabili si possono usare array con indice o numeri immediati come sorgente. Non è possibile salvare il risultato direttamente in un numero immediato (&n;&).

7 · Operatori Logici e di Confronto

Le funzioni di confronto restituiscono 0 (uguale/vero) o 1 (diverso/falso). Vengono usate come condizioni in if_, for_ e while_. Possono essere concatenate con && (AND) o || (OR).

Operatore	Sintassi	Ritorna 0 se...	Ritorna 1 se...
== uguaglianza int	== &i;&v1;& &i;&v2;&	v1 == v2	v1 != v2
s== uguaglianza str	s== &s;&str1;& &s;&str2;&	str1==str2	str1!=str2
! diverso	! &i;&v1;& &i;&v2;&	v1 != v2	v1 == v2
> maggiore	> &i;&v1;& &i;&v2;&	v1 > v2	v1 <= v2
< minore	< &i;&v1;& &i;&v2;&	v1 < v2	v1 >= v2

8 · Strutture di Controllo del Flusso

```
if_( ) {
    // corpo
}
else_ {
    // corpo alternativo
}
```

La condizione usa gli operatori della Sezione 7. Più condizioni si concatenano con && (AND logico) o || (OR logico). I blocchi if_ e else_ non terminano con '!'.

```
for_( , ) {
    // corpo
}
```

Esempio: for_(< &i;& &n&10& , + &i;& &n&1&) — esegue il corpo finché i < 10 e incrementa i di 1 ad ogni ciclo.

```
while_( ) {
    // corpo
}
```

```
}
```

Ripete il corpo finché la condizione è verificata. La condizione è identica a quella di `if_`.

■ `if_`, `for_`, `while_` e `else_` sono definiti nella sintassi ma NON ancora implementati nell'interprete corrente (versione 1.0).

9 · Dichiarazione e Chiamata di Funzioni

```
_nomefunzione(arg1, arg2) {  
    // corpo  
    return NULL ;  
    // oppure:  
    return &i;&var;& ;  
    // oppure:  
    return &n;&42& ;  
}
```

Le funzioni iniziano con un solo underscore. La funzione speciale `_start(arg)` è il punto di ingresso del programma. Il `return` può restituire NULL, una variabile o un numero immediato. Se viene restituito NULL la funzione può essere chiamata senza salvare il valore.

9.2 Chiamata di Funzione — tre forme

Forma	Sintassi	Quando usarla
Senza valore di ritorno	<code>__nome(arg1, arg2) :</code>	La funzione non restituisce niente (return NULL) o il valore non interessa
Un valore di ritorno con =	<code>= &i;&var;& __nome(arg) :</code>	La funzione restituisce un singolo valore intero da salvare in var
Più valori di ritorno	<code>__nome(arg) --&i;&v0;& --&i;&v1;& --&i;&v2;& :</code>	La funzione restituisce più valori; ogni -- indica una variabile di output

■ Le chiamate di funzione usano il doppio underscore `'__'`. La dichiarazione usa il singolo underscore `'_'`.

10 · Input / Output

Tutte le istruzioni I/O terminano con `:`. Le varianti `println_` e `lprintln_` aggiungono un newline rispettivamente dopo e prima della stampa.

Istruzione	Sintassi	Effetto
<code>print_</code>	<code>print_ &i;&var;& :</code>	Stampa il valore intero di var
<code>print_</code>	<code>print_ &l;&var;& :</code>	Stampa il valore float di var
<code>print_</code>	<code>print_ &c;&var;& :</code>	Stampa il carattere var
<code>print_</code>	<code>print_ &s;[]&str;& :</code>	Stampa l'intera stringa str
<code>print_</code>	<code>print_ &s;[N]&str;& :</code>	Stampa il carattere str[N]
<code>print_</code>	<code>print_ &i;[N]&arr;& :</code>	Stampa arr[N] (intero)

Istruzione	Sintassi	Effetto
print_	print_ &s;"testo"& :	Stampa il testo letterale
println_	println_ &i;&var;& :	Stampa var e poi va a capo
Inprint_	Inprint_ &s;[]&str;& :	Va a capo e poi stampa str
Inprintln_	Inprintln_ &i;&var;& :	Va a capo, stampa, va a capo
scan_	scan_ &i;&var;& :	Legge un intero da stdin in var
scan_	scan_ &s;[]&str;& :	Legge una stringa da stdin in str
scan_	scan_ &c;&var;& :	Legge un char da stdin in var

■ scan_ è definita nella sintassi ma NON ancora implementata nell'interprete v1.0.

11 · Direttive di Sistema

Le direttive di sistema iniziano con # e terminano con :. Gestiscono import, debug e controllo dell'interprete.

Direttiva	Sintassi	Stato	Descrizione
#import_	#import_ &f;&libreria.Zlib;& :	■ parziale	Importa (concatena) il codice sorgente della libreria
#debug_dt	#debug_dt :	✓ implementata	Attiva la modalità debug (stampa ogni istruzione analizzata)
#debug_df	#debug_df :	✓ implementata	Disattiva la modalità debug

12 · Esempio Completo Commentato

Il seguente programma dimostra le principali funzionalità del linguaggio Z: dichiarazione di variabili e array, assegnazione di valori, operazioni su char e int, e chiamata di funzione con valore di ritorno.

```
// Funzione che restituisce la somma di due argomenti
_calcola( &i;[5]&arr;& , &i;&v;& ) {
    int_ &i;&risultato;& :           // dichiara var locale
    = &i;&risultato;& &n;&67& :       // assegna 67
    return &i;&risultato;& :
}

// Punto di ingresso del programma
_start() {
    // --- Variabili intere ---
    int_ &i;&v0;& :                   // v0 = 0
    int_ &i;[5]&arr5;& :              // array di 5 interi

    = &i;&v0;& &n;&7& :                 // v0 = 7
    print_ &i;&v0;& :                 // stampa: 7
    println_ &s;" valore v0"& :

    = &i;[0]&arr5;& &n;&9& :            // arr5[0] = 9
    = &i;&v0;& &i;[0]&arr5;& :          // v0 = arr5[0] → v0 = 9

    // --- Variabili char ---
}
```

```

char_ &c;&ch0;& : // ch0 = '\0'
char_ &s;&[5]&str5;& : // array di 5 char

= &c;&ch0;& &k;&H;& : // ch0 = 'H'
= &s;&[0]&str5;& &k;&L;& : // str5[0] = 'L'
print_ &c;&ch0;& : // stampa: H
print_ &s;&[0]&str5;& : // stampa: L
lnprint_ &s;&" "& :

// --- Chiamata funzione con ritorno ---
= &i;&v0;& __calcola( &i;&[5]&arr5;& , &i;&v0;& ) :
print_ &i;&v0;& : // stampa: 67
println_ &s;&" risultato calcola"& :

}

```

13 · Riepilogo Rapido della Sintassi

Costrutto	Pattern sintattico	Termina con
Dichiarazione int var	int_ &i;&nome;&	:
Dichiarazione int array	int_ &i;&[dim]&nome;&	:
Dichiarazione char var	char_ &c;&nome;&	:
Dichiarazione char array	char_ &s;&[dim]&nome;&	:
Assegnazione	= &tipo;&_d&dest;& &tipo;&_s&src;&	:
Somma 3 op.	= + &i;&dest;& &i;&a;& &i;&b;&	:
Confronto uguaglianza	== &i;&v1;& &i;&v2;&	:
If	if_(condizione) { ... }	} (no :)
For	for_(cond , incr) { ... }	} (no :)
While	while_(cond) { ... }	} (no :)
Dichiarazione funzione	_nome(args) { ... }	} (no :)
Chiamata senza ritorno	__nome(args)	:
Chiamata con = ritorno	= &i;&var;& __nome(args)	:
Stampa	print_ &tipo;&nome;&	:
Stampa + newline	println_ &tipo;&nome;&	:
Newline + stampa	lnprint_ &tipo;&nome;&	:
Input	scan_ &tipo;&nome;&	:
Import libreria	#import_ &f;&file.Zlib;&	:
Debug on/off	#debug_dt / #debug_df	:

■ **Regola d'oro:** tutto termina con ':' tranne le aperture '{' e le chiusure '}'.